



SIMATS
ENGINEERING



SIMATS
Saveetha Institute of Medical And Technical Sciences
(Declared as Deemed to be University under Section 3 of UGC Act 1956)

From Messy Data to Meaningful Data

DSA0503 - Query Processing for Data Science

Guided by:
Dr. C. Anuradha

Presented By:
Chalamala Sai Harshitha(192424346)
A. Pavithra(192424347)
V. Tejaswini(192424360)

Messy Data

Data that is **not clean, consistent, or properly organized**, making it difficult to process and analyse.

➤ Key Characteristics

- **Incomplete** – Missing information
- **Inconsistent** – Different formats or representations
- **Incorrect** – Contains errors
- **Unorganized** – Poorly structured
- **Redundant** – Repeated information

Messy data can result in:

- Inaccurate analysis
- Unreliable results
- Incorrect decisions

Types of Data Problems

The main problems commonly found in messy data are:

Problem	Example
Missing Values	Age = NaN
Duplicate Records	Same customer recorded twice
Inconsistent Data	Ravi, ravi, RAVI
Invalid Values	Marks = 250
Inconsistent Formats	10/01/2025, 2025-01-10
Extra Spaces	" Ravi "

Impact of These Problem:

- These issues can cause **incorrect calculations, unreliable analysis, and misleading results.**

Sample Messy Dataset

Order ID	Customer	Product	Quantity	Price	City	Date
1001	Ravi Kumar	Laptop	1	50000	Chennai	10/01/2025
1002	Anitha	Keyboard	2	1200	chennai	2025-01-10
1003	Priya	Mobile	2	20000	Bangalore	11/01/2025
1004	Arun	Headphones	3	1500	CHENNAI	12/01/2025
1005	Priya	Mobile	2	20000	Bangalore	11/01/2025
1006	Meena	Laptop	—	55000	Chennai	13/01/2025
1007	Kiran	laptop	1	55000	Chennai	2025/01/14
1008	Suresh	Mobile	1	-20000	Delhi	15/01/2025
1009	Rahul	Mouse	2	800	Delhi	16-01-2025
1010	Divya	keyboard	1	1200	DELHI	2025/01/17
1009	Rahul	Mouse	2	800	Delhi	16-01-2025

Problems Identified

- **Missing value:** Quantity for Meena
- **Extra spaces:** Rahul's name
- **Duplicate record:** Order 1009
- **Different date formats**
- **Inconsistent text format**
- **Invalid value:** Price = -20000

Loading the Dataset Using Python

We use **Pandas** to load the messy sales dataset into a DataFrame

```
import pandas as pd
```

```
# Load the dataset
```

```
df = pd.read_csv("messy_sales_data.csv")
```

```
# Display the dataset
```

```
print(df)
```

	Order_ID	Customer	Product	Quantity	Price	City	Date
0	1001	Ravi Kumar	Laptop	1.0	50000	Chennai	10/01/2025
1	1002	Anitha	Keyboard	2.0	1200	chennai	2025-01-10
2	1003	Priya	Mobile	2.0	20000	Bangalore	11/01/2025
3	1004	Arun	Headphones	3.0	1500	CHENNAI	12/01/2025
4	1005	Priya	Mobile	2.0	20000	Bangalore	11/01/2025
5	1006	Meena	Laptop	NaN	55000	Chennai	13/01/2025
6	1007	Kiran	laptop	1.0	55000	Chennai	2025/01/14
7	1008	Suresh	Mobile	1.0	-20000	Delhi	15/01/2025
8	1009	Rahul	Mouse	2.0	800	Delhi	16-01-2025
9	1010	Divya	keyboard	1.0	1200	DELHI	2025/01/17
10	1009	Rahul	Mouse	2.0	800	Delhi	16-01-2025

What does the code do?

- `import pandas as pd` → Imports the Pandas library.
- `pd.read_csv()` → Reads the CSV file.
- `df` → Stores the dataset as a DataFrame.
- `print(df)` → Displays the dataset.

Data Cleaning Using Pandas

```
# Display basic information  
print(df.info())
```

```
# Check missing values  
print(df.isnull().sum())
```

```
# Check duplicate records  
print(df.duplicated().sum())
```

```
# View statistical information  
print(df.describe())
```

```
-----  
#   Column   Non-Null Count  Dtype  
-----  
0   Order_ID  11 non-null      int64  
1   Customer  11 non-null      object  
2   Product    11 non-null      object  
3   Quantity   10 non-null      float64  
4   Price      11 non-null      int64  
5   City       11 non-null      object  
6   Date       11 non-null      object  
dtypes: float64(1), int64(2), object(4)  
memory usage: 748.0+ bytes
```

Missing Values:

```
Order_ID    0  
Customer    0  
Product     0  
Quantity    1  
Price       0  
City        0  
Date        0  
dtype: int64
```

Duplicate Records:

```
1
```

Statistical Summary:

	Order_ID	Quantity	Price
count	11.000000	10.000000	11.000000
mean	1005.818182	1.700000	16863.636364
std	3.060006	0.674949	25745.301425
min	1001.000000	1.000000	-20000.000000
25%	1003.500000	1.000000	1000.000000
50%	1006.000000	2.000000	1500.000000
75%	1008.500000	2.000000	35000.000000
max	1010.000000	3.000000	55000.000000

Handling Missing Values

```
# Check missing values
print(df.isnull().sum())
```

```
# Fill missing Quantity with the median
df['Quantity'] = df['Quantity'].fillna(
    df['Quantity'].median()
)
```

Missing values before cleaning:

1

Dataset after handling missing values:

	Order_ID	Customer	Product	Quantity	Price	City	Date
0	1001	Ravi Kumar	Laptop	1.0	50000	Chennai	10/01/2025
1	1002	Anitha	Keyboard	2.0	1200	chennai	2025-01-10
2	1003	Priya	Mobile	2.0	20000	Bangalore	11/01/2025
3	1004	Arun	Headphones	3.0	1500	CHENNAI	12/01/2025
4	1005	Priya	Mobile	2.0	20000	Bangalore	11/01/2025
5	1006	Meena	Laptop	2.0	55000	Chennai	13/01/2025
6	1007	Kiran	laptop	1.0	55000	Chennai	2025/01/14
7	1008	Suresh	Mobile	1.0	-20000	Delhi	15/01/2025
8	1009	Rahul	Mouse	2.0	800	Delhi	16-01-2025
9	1010	Divya	keyboard	1.0	1200	DELHI	2025/01/17
10	1009	Rahul	Mouse	2.0	800	Delhi	16-01-2025

Removing Duplicate Data

```
# Find duplicate records  
print(df[df.duplicated()])
```

Duplicate records: 1

```
# Remove duplicate records  
df = df.drop_duplicates()
```

```
# Check the result  
print(df)
```

Dataset after removing duplicates:

	Order_ID	Customer	Product	Quantity	Price	City	Date
0	1001	Ravi Kumar	Laptop	1.0	50000	Chennai	10/01/2025
1	1002	Anitha	Keyboard	2.0	1200	chennai	2025-01-10
2	1003	Priya	Mobile	2.0	20000	Bangalore	11/01/2025
3	1004	Arun	Headphones	3.0	1500	CHENNAI	12/01/2025
4	1005	Priya	Mobile	2.0	20000	Bangalore	11/01/2025
5	1006	Meena	Laptop	2.0	55000	Chennai	13/01/2025
6	1007	Kiran	laptop	1.0	55000	Chennai	2025/01/14
7	1008	Suresh	Mobile	1.0	-20000	Delhi	15/01/2025
8	1009	Rahul	Mouse	2.0	800	Delhi	16-01-2025
9	1010	Divya	keyboard	1.0	1200	DELHI	2025/01/17

Standardizing Text Data

```
# Remove extra spaces and standardize text
```

```
df['Customer'] = df['Customer'].str.strip().str.title()
```

```
df['Product'] = df['Product'].str.strip().str.title()
```

```
df['City'] = df['City'].str.strip().str.title()
```

```
# Standardize date format
```

```
df['Date'] = pd.to_datetime(
```

```
    df['Date'],
```

```
    format='mixed',
```

```
    dayfirst=True
```

```
).dt.strftime('%Y-%m-%d')
```

Dataset after standardization:

	Order_ID	Customer	Product	Quantity	Price	City	Date
0	1001	Ravi Kumar	Laptop	1.0	50000	Chennai	2025-01-10
1	1002	Anitha	Keyboard	2.0	1200	Chennai	2025-01-10
2	1003	Priya	Mobile	2.0	20000	Bangalore	2025-01-11
3	1004	Arun	Headphones	3.0	1500	Chennai	2025-01-12
4	1005	Priya	Mobile	2.0	20000	Bangalore	2025-01-11
5	1006	Meena	Laptop	2.0	55000	Chennai	2025-01-13
6	1007	Kiran	Laptop	1.0	55000	Chennai	2025-01-14
7	1008	Suresh	Mobile	1.0	-20000	Delhi	2025-01-15
8	1009	Rahul	Mouse	2.0	800	Delhi	2025-01-16
9	1010	Divya	Keyboard	1.0	1200	Delhi	2025-01-17

Handling Invalid Values

```
# Find invalid prices
print(df[df['Price'] < 0])
```

```
# Replace negative prices with the median price
median_price = df[df['Price'] >= 0]['Price'].median()
```

```
df.loc[df['Price'] < 0, 'Price'] = median_price
```

Invalid prices:

	Order_ID	Customer	Product	Quantity	Price	City	Date
7	1008	Suresh	Mobile	1.0	-20000	Delhi	2025-01-15

Dataset after handling invalid prices:

	Order_ID	Customer	Product	Quantity	Price	City	Date
0	1001	Ravi Kumar	Laptop	1.0	50000	Chennai	2025-01-10
1	1002	Anitha	Keyboard	2.0	1200	Chennai	2025-01-10
2	1003	Priya	Mobile	2.0	20000	Bangalore	2025-01-11
3	1004	Arun	Headphones	3.0	1500	Chennai	2025-01-12
4	1005	Priya	Mobile	2.0	20000	Bangalore	2025-01-11
5	1006	Meena	Laptop	2.0	55000	Chennai	2025-01-13
6	1007	Kiran	Laptop	1.0	55000	Chennai	2025-01-14
7	1008	Suresh	Mobile	1.0	20000	Delhi	2025-01-15
8	1009	Rahul	Mouse	2.0	800	Delhi	2025-01-16
9	1010	Divya	Keyboard	1.0	1200	Delhi	2025-01-17

Clean Dataset

```
print("Final Clean Dataset:")  
print(df)
```

Final Clean Dataset:

	Order_ID	Customer	Product	Quantity	Price	City	Date
0	1001	Ravi Kumar	Laptop	1.0	50000	Chennai	2025-01-10
1	1002	Anitha	Keyboard	2.0	1200	Chennai	2025-01-10
2	1003	Priya	Mobile	2.0	20000	Bangalore	2025-01-11
3	1004	Arun	Headphones	3.0	1500	Chennai	2025-01-12
4	1005	Priya	Mobile	2.0	20000	Bangalore	2025-01-11
5	1006	Meena	Laptop	2.0	55000	Chennai	2025-01-13
6	1007	Kiran	Laptop	1.0	55000	Chennai	2025-01-14
7	1008	Suresh	Mobile	1.0	20000	Delhi	2025-01-15
8	1009	Rahul	Mouse	2.0	800	Delhi	2025-01-16
9	1010	Divya	Keyboard	1.0	1200	Delhi	2025-01-17

Data Transformation

```
# Calculate sales amount for each order
```

```
df['Sales_Amount'] = df['Quantity'] * df['Price']
```

```
print(df[['Order_ID', 'Product', 'Sales_Amount']])
```

Dataset after transformation:

	Order_ID	Customer	Product	Quantity	Price	City	Date
0	1001	Ravi Kumar	Laptop	1.0	50000	Chennai	2025-01-10
1	1002	Anitha	Keyboard	2.0	1200	Chennai	2025-01-10
2	1003	Priya	Mobile	2.0	20000	Bangalore	2025-01-11
3	1004	Arun	Headphones	3.0	1500	Chennai	2025-01-12
4	1005	Priya	Mobile	2.0	20000	Bangalore	2025-01-11
5	1006	Meena	Laptop	2.0	55000	Chennai	2025-01-13
6	1007	Kiran	Laptop	1.0	55000	Chennai	2025-01-14
7	1008	Suresh	Mobile	1.0	20000	Delhi	2025-01-15
8	1009	Rahul	Mouse	2.0	800	Delhi	2025-01-16
9	1010	Divya	Keyboard	1.0	1200	Delhi	2025-01-17

Sales_Amount

0	50000.0
1	2400.0
2	40000.0
3	4500.0
4	40000.0
5	110000.0
6	55000.0
7	20000.0
8	1600.0
9	1200.0

Data Analysis Using Pandas

```
# Total sales
total_sales = df['Sales_Amount'].sum()

# Average sales per order
average_sales = df['Sales_Amount'].mean()

# Sales by product
sales_by_product = df.groupby('Product')['Sales_Amount'].sum()

print("Total Sales:", total_sales)
print("\nAverage Sales per Order:", average_sales)
print("\nSales by Product:")
print(sales_by_product)
```

```
Total Sales: 324700.0
```

```
Average Sales per Order: 32470.0
```

```
Sales by Product:
```

```
Product
```

```
Headphones      4500.0
```

```
Keyboard        3600.0
```

```
Laptop         215000.0
```

```
Mobile         100000.0
```

```
Mouse          1600.0
```

```
Name: Sales_Amount, dtype: float64
```

Result

After cleaning and analyzing the sales dataset, we were able to:

- Remove duplicate records
- Handle missing values
- Standardize inconsistent text
- Correct invalid values
- Standardize date formats
- Calculate Sales Amount
- Find total sales
- Compare sales by product
- Identify the highest-selling product

Thank You